

VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
PROGRAMŲ SISTEMŲ STUDIJŲ PROGRAMA

**Hibridinio genetinio paieškos algoritmo transporto
maršrutų optimizavimo uždaviniams spręsti
lygiagretinimas**

**Parallelization of Hybrid Genetic Search Algorithm for Solving
Vehicle Routing Problem**

Kursinis darbas

Atliko: 4 kurso 1 grupės studentas
Domantas Keturakis

Darbo vadovas: Doc., Dr. Algirdas Lančinskas

Vilnius – 2025

Turinys

Terminai	3
Įvadas	4
Transporto maršrutų optimizavimo uždaviniai	5
Tikslūs ir apytiksliai metodai	5
VRP variacijos	5
CVRP	5
HGS algoritmo veikimas	6
Literatūros analizė	7
Lygiagretinimas	9
Lygiagretintos ir nuoseklios programos palyginimas	10
Metodika	10
Rezultatai	11
Rezultatai ir išvados	12
Rezultatai	12
Išvados	12
Priedai	13
Vidutinės pasiektos sprendimų kainos ir tarpas galutiniu laiko momentu . .	13
Šaltiniai	17

Terminai

- Populiacija – Rinkinys invididų.
- Individas – Užduoties sprendinys t.y. rinkinys maršrutų.
- VRP – Maršrutų optimizavimo uždavinys (*angl. Vehicle Routing Problem*).
- CVRP – *angl. Capacitated Vehicle Routing Problem*. Kiekviena transporto priemonė turi maksimalią siuntų talpą.
- VRPTW – *angl. VRP with Time Windows*.
- GVRP – *angl. Generalized VRP*. Klientai grupuojami į klusterius. Tik vienas klientas iš viso klusterio turi būti aplankytas.
- CluVRP – *angl. Clustered VRP*. Klientai grupuojami į klusterius. Visi klientai klusteryje turi būti aplankyti prieš važiuojant į kitą klusterį.
- SoftCluVRP – *angl. Clustered VRP*. Klientai grupuojami į klusterius. CluVRP variantas, kuriame į klusterį leidžiama aplankyti kelis kartus.
- MDVRP – *angl. Multidepot VRP*.
- PVRP – *angl. Periodic VRP*. Pridedama laiko dimescija, sprendinys susidaro iš kelių maršrutų rinkinių atitinkamus dienas, kuriomis bus aplankomi klientai.
- MDPVRP – *angl. Multidepot Periodic VRP*. MDVRP ir PVRP kombinacija.
- CVRPPD – *angl. CVRP Pickup and Delivery*. CVRP ir VRPPD kombinacija.

Įvadas

VRP – Transporto maršrutų optimizavimo uždavinys (*angl. Vehicle Routing Problem*) yra uždavinys, kurio tikslas yra surasti kuo optimaliausią maršrutų rinkinį Lygt. 1. Optimaliai parinkti maršrutai gali lemti kiek klientų įmanoma aplankyti per nustatytą laiką, sumažinti transporto kaštus. Pirmą kartą ši problema aprašyta [DR59], kur autorius aprašė algoritmą, kuris suranda optimalius maršrutus tarp kuro depo ir degalinių. Tai yra modernios logistikos optimizavimo uždavinys – optimaliai parinkti maršrutai gali lemti mažesnius kainos ir pristatymo laiko kaštus.

Kur keliaujančio pardavėjo uždavinyje pagrindinė užduotis yra surasti optimaliausią kelią vienam keliautojui – pardavėjui, VRP sprendiniai susidaro iš kelių keliautojų – literatūroje dažnai tiesiogiai vadinama transporto priemonėmis.

Hibridinis genetinis paieškos (*angl. Hybrid Genetic Search – HGS*) – yra vienas iš efektyviausių genetinių metaheuristinių algoritmų [Pet+23]. Šis algoritmas ir vėlesnės pagerintos versijos išlieka etalonas daugeliui VRP variantų, „DIMACS“ konkurse [DIM22] parodęs geriausius rezultatus VRPTW uždavinyje [Koo+22], ir kurio modifikuotas variantas [Jia+22] pasirodė geriausiai CVRP uždavinyje. Šis algoritmas yra pritaikytas CVRP, VRPTW, GVRP [Lat25a], CluVRP, SoftCluVRP [Lat25b].

Šio **darbo tikslas** – išlygiagretinti hibridinio genetinio paieškos algoritmą, skirtą transporto maršrutų optimizavimo uždaviniams spręsti, siekiant sumažinti vykdymo laiką neprarandant ar net pagerinant sprendinių kokybę.

Uždaviniai:

1. Išsirinkti duomenų rinkinį pagal, kurį galima būtų testuoti/analizuoti sprendinius.
2. Išanalizuoti, kaip veikia HGS algoritmas
3. Atrinkti paralelizuojamas dalis, ar dalis, kurias galima pakeisti paralelizuojamomis.
4. Palyginti rezultatus su kitais **state-of-the-art** algoritmais

Transporto maršrutų optimizavimo uždaviniai

Tikslūs ir apytiksliai metodai

Nors egzistuoja įrankiai, kurie pasiteklia tikslūs metodai (pavyzdžiui „Google or-tools“ [PF25]), kadangi šis uždavinys priklauso *NP-Hard* sudėtingumo klasei, visgi dominuoja heuristikomis ir metaheuristikomis grįsti algoritmai, nes šie beveik optimalius sprendinius suranda per greitesnį laiko tarpą sunaudodami mažiau resursų. Tikslūs metodai su ypač dideliais kiekiais duomenų tampa nepraktiški. Metaheuristiciniai algoritmai išsiskiria šioje uždavinių klasėje kaip efektyviausi, pasižymintys žemu algoritmo vykdymo laiku ir aukšta uždavinių rezultatų kokybe.

Šiai problemai egzistuoja heuristiniai ir metaheuristiciniai (Aukštesnio lygio strategija, kuris diriguoja, kurias heuristikas pritaikyti, kad efektyviau atrasti sprendinius) algoritmai. Keli dominuojantys pavyzdžiai [Ada+24]

- „Adaptive Large Neighborhood Search“ ir „Hybrid Adaptive Large Neighborhood Search“
- „Hybrid Genetic Search (HGS)“
- „Simulated Annealing Algorithm (SAA)“
- „Ant colony optimization (ACO)“

VRP variacijos

Praktikoje taikomos keletas VRP variacijų (CVRP, VRPTW, MDPVRP, PVRP ir kt.). Jos įveda papildomus apribojimus maršrutų ilgiui, transporto priemonių panaudojimo laikui ir talpai, ar prideda papildomas salygas:

- naudojamos transporto priemonės turi limituotą talpą (CVRP);
- visi klientai turi gali būti aplankyti tik specifinėmis darbo valandomis (VRPTW);
- keli depai iš kurių galima pradėti maršrutą (MDVRP);
- maršrutai planuojami per kelias dienas, t.y. vieni klientai gali būti aplankyti vieną dieną, o kiti kitą. (Periodic VRP);
- kt.

Šis darbas atsižvelgia tik į CVRP uždavinį.

CVRP

CVRP nagrinėjamas grafas $G = (V, E)$, kuriame $v_0 \in V$ žymi depą, kuris turi m transporto priemonių, o likusios viršūnės $\{v_1, \dots, v_{|V|}\}$ atitinka klientus, kuriuos reikia aplankyti. Kiekviena briauna $(i, j) \in E$ reiškia galimybę keliauti tarp vietų i ir j su kaina $c_{i,j}$ – atstumas tarp vietų i ir j . CVRP reikia surasti sprendinį, kuriame panaudotos ne daugiau kaip K transporto priemonių, prasidedančių ir pasibaigiančių depe, taip, kad kiekvienas klientas būtų aplankytas vieną kartą ir bendras klientų paklausos dydis bet kuriame maršrute neviršytų transporto priemonės talpos Q , o bendras transporto priemonių nuvažiuotas atstumas – kaina (žr. 1-a lygtis) kiek įmanoma mažesnis.

$$\text{Sprendinio kaina} = \sum_{k=1}^K \sum_{i=0}^{|V|} \sum_{j=0}^{|V|} c_{i,j} x_{i,j,k}$$

$c_{i,j}$ = astumas nuo kliento i iki kliento j
 $x_{i,j,k}$ = 1 Indikatorinė funkcija, kuri
 lygi 1, jei transporto priemonė k ($1 \leq k \leq K$)
 keliauja nuo kliento i iki kliento j ,
 lygi 0 priešingu atveju

(1)

Lyginant transporto maršrutų optimizavimo uždavinio sprendinius taip pat naudojamas tarpas (*angl. gap*), kuris nusako atstumą nuo geriausio sprendinio išreikštas procentais Lygt. 2.

$$\text{Tarpas} = \left(\frac{Z_s - Z_{\text{BKS}}}{Z_{\text{BKS}}} \right) \cdot 100\%$$

Z_s = Pasirinkto algoritmo sprendinio kaina
 Z_{BKS} = Geriausio sprendinio kaina

(2)

HGS algoritmo veikimas

Pirma aprašytas "A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems" [Vid+12] (2012) skirtas spręsti MDPVRP. Patobulintas per daugelį iteracijų: [Vid+14], [Vid+16], [Vid17], [Vid+21], [Vid22] ir pritaikytas CVRP.

Genetiniai algoritmai imituoja evoliucijos procesą. Populiacija yra aibė, kurią sudaro individai (t.y. užduoties sprendiniai). Šie algoritmai naudoja įvairius kryžminimo operatorius, kurie iš kelių individų populiacijoje sukuria naują, mutuotą individą ir prideda prie populiacijos. Prastos kokybės ir panašūs individai (sprendiniai) vykdymo eigoje yra pašalinami iš populiacijos.

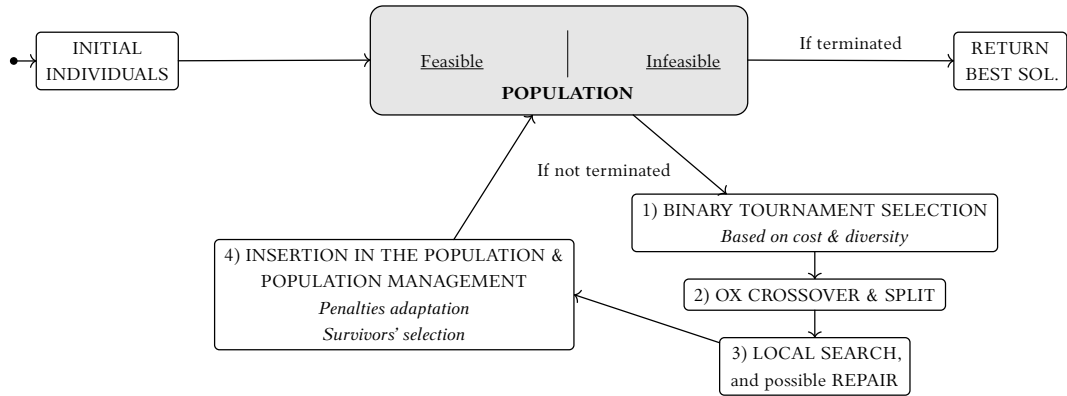
Genetinis hybridinis paieškos algoritmas prie genetinio komponento prideda pagerinimo žingsnį (vietinę paiešką (*angl. local search*)), kuri po kryžminimo žingsnio yra pritaikoma naujam individui, kad pagerinti gautą individo kokybę.

Vietinėje paieškoje taikomos *relocate*, *2-opt*, *2-opt**, bei *swap** heuristikos.

Palaikant neįvykdomus (*angl. infeasible*) ir įvykdomus (*angl. feasible*) sprendinius, palaikoma populiacijos įvairovė (*angl. diversity*), kuri leidžia išvengti lokalios minimos (*angl. local minima*) iteruojant per sprendinius.

- 1 Sugeneruoti pradinę populiaciją ir pagerinti ją vietine paieška
- 2 **Kol** iteracijų be pagerėjimo skaičius mažesnis už ribą ir $t < T_{\text{max}}$ atlikti
- 3 Pasirinkti tėvinius individus (dvejainis turnyras (*angl. binary tournament*))
- 4 Atlikti kryžminimą (*angl. crossover*)
- 5 Išmokyti naują individą (vietinė paieška)
- 6 Įterpti išmokytą individą į atitinkamą subpopuliaciją
- 7 **Jeigu** individas neįvykdomas:
- 8 Su 50% tikimybe bandyti sutaisyti individą ir įtraukti į atitinkamą subpopuliaciją
- 9 **Jeigu** pasiektas maksimalus aibės dydis:
- 10 pašalinti blogiausius ir neįvairius individus iš populiacijos
- 11 Patikslinti baudos parametrus (*angl. penalty parameters*)
- 12 Grąžinti geriausią įvykdomą individą

Pav. 1: HGS algoritmo pseudokodas [Vid+12]



Pav. 2: HGS veikimas [Vid22]

TODO: Išversti į lietuvių k.???

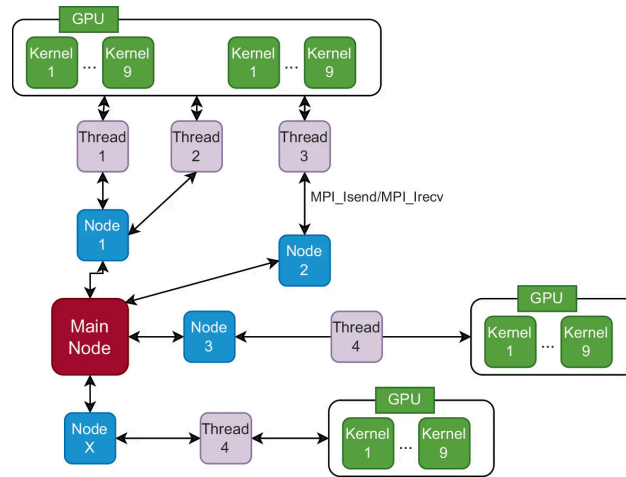
Literatūros analizė

[LHW25] lygiagrelinimui vietinės paieškos algoritmą išreiškia tenzorių operatoriais, tai leidžia HGS vykdymą perkelti ant GPU. Taip pagreitintas vietinės paieškos operatorius. Tačiau [LHW25] pasiūlytas metodas nėra pritaikomas HGS su swap*. „Dabartinė sprendinių reprezentacija per tensorius neleidžia lengvai įgyvendinti apkarpymo strategijų kaimynysčių sumažinimo technikų, kurie dažnai yra naudojami vietinės paieškos grįžtais algoritmais.“¹

[SND23] HGS pritaiko CVRPPD, perkelia tėvų pasirinkimo (*angl. selection*), kryžminimo (*angl. crossover*) ir taisymo (*angl. repair*) žingsnius į atskiras gijas. Kiekviena gija papildomai atlieka vietinę paiešką (*2-opt*, *relocate*, *swap*) pasitelkiant GPU, tačiau nepasitelkia *swap** heuristiką, kuri pagal [Vid22] padeda surasti aukštesnės kokybės sprendinius.

Priešintai nei dauguma implementacijų [Mun+23] naudoja grafų duomenų struktūras, panaudojami tik *2-opt* ir arčiausio kaimyno heuristikos (*angl. nearest-neighbor*). Šios heuristikos pritaikytos vykdymui GPU aplinkoje naudojant CUDA.

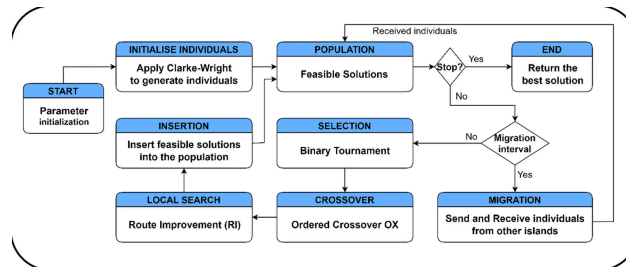
¹the current design of the tensor representation of solutions doesn't support easy implementation of pruning strategies and neighborhood reduction techniques that are often used in local search-based routing algorithms.



Pav. 3: HGS lygiagretintas [SND23]

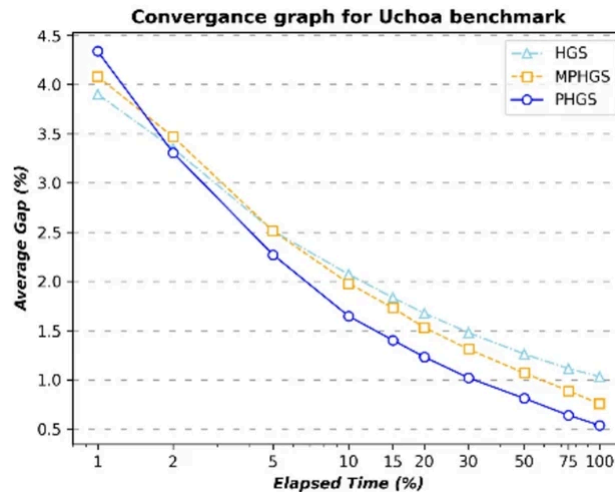
[YT21] pasitelkia GPU lygiagretinimui. Kiekvienam maršrutui skiriama atskira GPU gija, kuri vykdo vietinės paieškos žingsnį naudojant GPU. Šiuo metodu pilnas resursų išnaudojimas priklauso nuo to ar sukurtų maršrutų skaičius sutampa su gijų skaičiumi. Atvejai, kai vietinės paieškos žingsniai modifikuoja kitų maršrutų sprendinių, gijos įrašo savo sprendinius į atskirą masyvą, kuris vėliau yra redukuojamas į vieną sprendinį, pasirenkant geriausią sprendinį. Analogiškai, vėlesnėje žingsnyje kiekvienam klientui priskiriama gija. Kiekviena gija apskaičiuoja pagerėjimą ar pablogėjimą apsikeistus vietą maršrute su kitu klientu.

[Jam+25] aprašo *PHGS* (angl. *Parallel Hybrid Genetic Search*), kur kombinuoja HGS su salų modeliu, aprašytu [Rez+24], kur kiekviena gija, vykdo tą patį HGS algoritmą, pridedamas individų migracijos žingsnis, kuris leidžia keistis sprendiniais tarp gijų.



Pav. 4: HGS su salų modeliu [Jam+25]

PHGS rodo vos ne du kartus geresnius rezultatus galutiniame laiko momente.



Pav. 5: Palyginimas tarp vidutinio tarpo (Y ašis) ir vykdymo laiko (X ašis) [Jam+25].

Lygiagretinimas

Paimta [Vid22] HGS algoritmo implementacija², kuri naudojama kaip pagrindas lygiagretinimui.

Daugiausiai laiko užima vietinės paieškos žingsnis [Jam+25], šio autoriaus atliktais matavimais vietinė paieška užima 85% vykdymo laiko. Todėl siekiant sumažinti viso HGS algoritmo vykdymo laiką šį žingsnį yra labiausiai verta lygiagretinti.

Lygiagretinimas įgyvendintas kiekvienai gijai, atliekant vietinę paiešką. Lygiagretinta HGS-CVRP versija pavaizduota Pav. 6. Nuosekliai veikiančioje sekcijoje parenkami skirtingi tėviniai individai, kryžminimo metu sukuriama kiekvienai gijai. Tada kiekviena gija lygiagrečiai atlieka vietinės paieškos žingsnį. Vėliau, kai kiekviena gija atlieka šį žingsnį, individai yra pridedami į visų populiaciją.

TODO: praplėsti ir padaryti diagramą

- 1 Sugeneruoti pradinę populiaciją ir pagerinti ją vietine paieška
- 2 **Kol** iteracijų be pagerėjimo skaičius mažesnis už ribą ir $t < T_{\max}$ atlikti
- 3 Pasirinkti $N/3 * 2$ tėvinius individus (dvejietinis turnyras (*angl. binary tournament*))
- 4 Atlikti kryžminimą (*angl. crossover*) N kartų
- 5 (Kiekvienoje gijoje) išmokyti naują individą
- 6 Įterpti išmokytą individą į atitinkamą subpopuliaciją
- 7 **Jeigu** individas neįvykdomas:
- 8 (Kiekvienoje gijoje) su 50% tikimybe bandyti sutaisyti individą ir įtraukti į atitinkamą subpopuliaciją
- 9 **Jeigu** pasiektas maksimalus aibės dydis:
- 10 pašalinti blogiausius ir neįvairius individus iš populiacijos
- 11 Patikslinti baudos parametrus (*angl. penalty parameters*)
- 12 Grąžinti geriausią įvykdomą individą

Pav. 6: HGS algoritmo pseudokodas [Vid+12]

Palyginimui naudoti geriausių sprendinių rinkinys⁴.

²Nuolatinė repozitoriją nuoroda <https://github.com/vidalt/HGS-CVRP/tree/1a927955cd2861a29d978f0d359d6e647db9319c>

³ N – gijų skaičius

⁴<https://galgos.inf.puc-rio.br/cvrplib/index.php/en/instances>, prieigos data: 2026-01-07

Lygiagretintos ir nuoseklios programos palyginimas

Metodika

HGS ir kiti iteratyvūs algoritmas sustoja tik, kai pasiekiamas tam tikras kriterijus. *HGS* atveju, tai iteracijų skaičius be pagerėjimo ar veikimo laikas. Kadangi šiek parametrai gali būti laisvai parinkti, pasirinkus pakankamai aukštus parametrus algoritmo veikimo laikas gali tęstis ilgiau negu dabartinis visatos amžius.

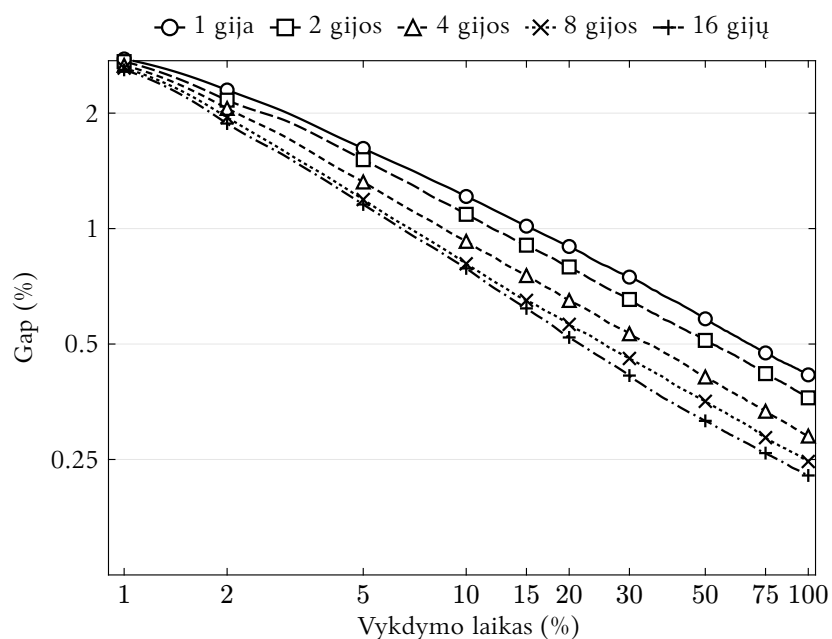
Dėl rezultatų palyginamumo pasirinkta naudoti [Vid22] aprašytus duomenų rinkinius ([Uch+17]) ir metodiką: „Mes stebime kiekvieno algoritmo pažangą iki laiko ribos $T_{\max} = n \cdot 240 / 100$ sekundžių, kur n reiškia klientų skaičių. Todėl mažiausias atvejis su 100 klientais vykdomas 4 minutes, o didžiausias atvejis su 1000 klientų vykdomas 40 minučių. Kiekvieno veikimo metu mes užregistruojame geriausią sprendimo vertę po 1%, 2%, 5%, 10%, 15%, 20%, 30%, 50%, 75% ir 100% laiko ribos, kad galėtume įvertinti algoritmų našumą skirtinguose paieškos etapuose [Vid22].“⁵

Vykdymo duomenys surinkti paleidžiant lygiagretintą ir palyginimui originalią *HGS-CVRP* programą ant „Intel® Xeon® Gold 6252“ procesoriaus. Šiam eksperimentui iteracijų skaičius begalinis iteracijų be pagreitėjimo skaičius ir $T_{\max} = n \frac{24}{100}$, 10 kartų mažesnis negu [Vid22], tam, kad sutilpti į duotus MIF STSC resursų limitus. Lygiagretintos programos versija patalpinta „Codeberg“ repozitorijoje⁶.

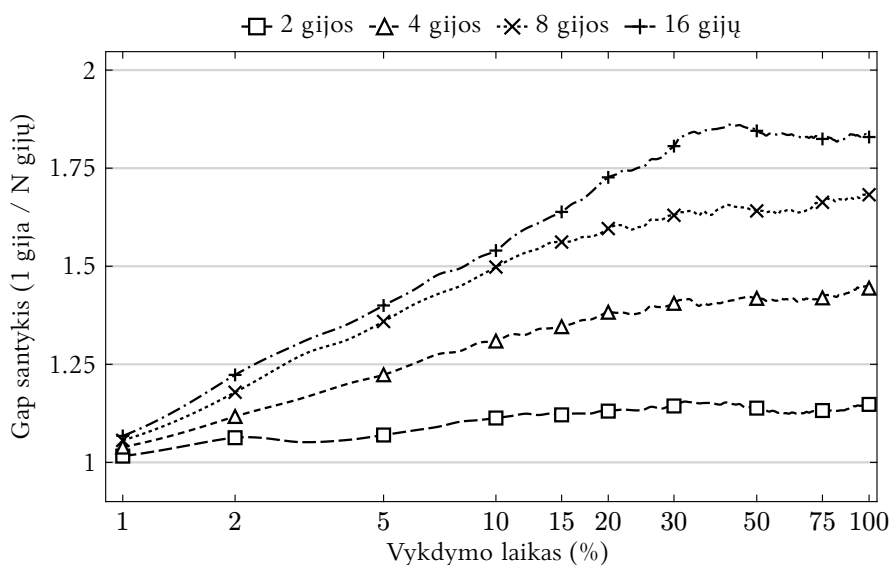
⁵We monitor each algorithm's progress up to a time limit of $T_{\max} = n \cdot 240 / 100$ seconds, where n represents the number of customers. Therefore, the smallest instance with 100 clients is run for 4 minutes, whereas the largest instance containing 1000 clients is run for 40 minutes. During each run, we record the best solution value after 1%, 2%, 5%, 10%, 15%, 20%, 30%, 50%, 75%, and 100% of the time limit to measure the performance of the algorithms at different stages of the search.

⁶<https://codeberg.org/Dom/HGS-CVRP/src/commit/411e391ffefac9a308d28e280194d65004d8332c>

Rezultatai



Pav. 7: Sprendimo kokybė pagal gijų skaičių. Gap apskaičiuojamas pagal BKS, o x ašis rodo vykdymo laiką procentais.



Pav. 8: Sprendimo kokybės santykis tarp viengijio ir N gijų sprendimų ($\text{Gap}(1 \text{ gija}) / \text{Gap}(N \text{ gijų})$) priklausomai nuo vykdymo laiko.

Iš rezultatų matyti, kad užduočių kokybė t.y. COST nesumažėjo, tačiau vykdymo laikas sumažėjo X kartų iki Y gijų skaičiaus, daugiau didinant gijų skaičių vykdymo laikas vidutiniškai pakilo X kartų

Palyginus su [Jam+25] matomas didesnis pagreitis su 16

Rezultatai ir išvados

Rezultatai

1. Išsirinkti duomenų rinkinį pagal, kurį galima būtų testuoti/analizuoti sprendinius.
 2. Išanalizuoti, kaip veikia HGS algoritmas
 3. Atrinkti paralelizuojamas dalis, ar dalis, kurias galima galima pakeisti paralelizuojamomis.
 4. Palyginti rezultatus su kitais state-of-the-art algoritmais
1. Parinktas duomenų rinkinys pagal, kurį galima būtų testuoti/analizuoti sprendinius.
 2. Atlinka HGS algoritmo veikimo analizė.
 3. Atrinkti paralelizuojamas dalis, ar dalis, kurias galima galima pakeisti paralelizuojamomis.
 4. Atliktas palyginimas su kitais state-of-the-art algoritmais.

Išvados

TODO

Priedai

Vidutinės pasiektos sprendimų kainos ir tarpas galutiniu laiko momentu

Uždavinys	1 gija ⁷		2 gijos		4 gijos		8 gijos		16 gijos	
	Avg	Gap	Avg	Gap	Avg	Gap	Avg	Gap	Avg	Gap
X-n101-k25	27591	0%	27591	0%	27591	0%	27591	0%	27591	0%
X-n106-k14	26415.6	0.2%	26402.6	0.15%	26396.8	0.13%	26386.6	0.09%	26392	0.11%
X-n110-k13	14971	0%	14971	0%	14971	0%	14971	0%	14971	0%
X-n115-k10	12747	0%	12747	0%	12747	0%	12747	0%	12747	0%
X-n120-k6	13332	0%	13332	0%	13332	0%	13332	0%	13332	0%
X-n125-k30	55541.8	0.01%	55540.4	0%	55541.8	0.01%	55539	0%	55540.4	0%
X-n129-k18	28940	0%	28940	0%	28940	0%	28940	0%	28940	0%
X-n134-k13	10924.8	0.08%	10916.8	0.01%	10916	0%	10919.4	0.03%	10916	0%
X-n139-k10	13590	0%	13590	0%	13590	0%	13590	0%	13590	0%
X-n143-k7	15702.2	0.01%	15700	0%	15700	0%	15700	0%	15700	0%
X-n148-k46	43448	0%	43448	0%	43448	0%	43448	0%	43448	0%
X-n153-k22	21225.6	0.03%	21227	0.03%	21225	0.02%	21225	0.02%	21225	0.02%
X-n157-k13	16876	0%	16876	0%	16876	0%	16876	0%	16876	0%
X-n162-k11	14138	0%	14138	0%	14138	0%	14138	0%	14138	0%
X-n167-k10	20561.2	0.02%	20557	0%	20557	0%	20557	0%	20557	0%
X-n172-k51	45607	0%	45607	0%	45607	0%	45607	0%	45607	0%
X-n176-k26	47812	0%	47812	0%	47813	0%	47812	0%	47812	0%
X-n181-k23	25572	0.01%	25569.8	0%	25569.6	0%	25569	0%	25569.8	0%
X-n186-k15	24145	0%	24145.4	0%	24145	0%	24145	0%	24145	0%
X-n190-k8	17059.6	0.47%	17079.6	0.59%	17008	0.16%	16996.8	0.1%	16992.8	0.08%
X-n195-k51	44234.6	0.02%	44235	0.02%	44236.8	0.03%	44229.8	0.01%	44238.2	0.03%
X-n200-k36	58618.2	0.07%	58624.4	0.08%	58603.8	0.04%	58604.8	0.05%	58598	0.03%
X-n204-k19	19568	0.02%	19565	0%	19565	0%	19565	0%	19565	0%
X-n209-k16	30677.4	0.07%	30672	0.05%	30666.6	0.03%	30657.2	0%	30663.6	0.02%
X-n214-k11	10879.2	0.21%	10872.8	0.15%	10870.8	0.14%	10868.4	0.11%	10867.4	0.11%
X-n219-k73	117609.6	0.01%	117603.8	0.01%	117605.2	0.01%	117597.2	0%	117600.8	0%
X-n223-k34	40458.4	0.05%	40481.8	0.11%	40475.4	0.09%	40482.2	0.11%	40458.2	0.05%
X-n228-k23	25743	0%	25746.8	0.02%	25749	0.03%	25743.6	0.01%	25743	0%
X-n233-k16	19235.2	0.03%	19243.8	0.07%	19239.6	0.05%	19248.4	0.1%	19237.8	0.04%
X-n237-k14	27054	0.04%	27043.6	0.01%	27044.4	0.01%	27042.4	0%	27042.4	0%
X-n242-k48	82917.4	0.2%	82846.2	0.12%	82865.6	0.14%	82908.6	0.19%	82865.6	0.14%
X-n247-k50	37380.8	0.29%	37357.4	0.22%	37346.6	0.19%	37322.4	0.13%	37335.2	0.16%
X-n251-k28	38752.6	0.18%	38751.4	0.17%	38752.8	0.18%	38756.8	0.19%	38766.4	0.21%
X-n256-k16	18874.4	0.19%	18880	0.22%	18881.6	0.23%	18880	0.22%	18880	0.22%

⁷Naudota originali realizacija

Uždavinys	1 gija ⁸		2 gijos		4 gijos		8 gijos		16 gijos	
	Avg	Gap	Avg	Gap	Avg	Gap	Avg	Gap	Avg	Gap
X-n261-k13	26642.2	0.32%	26607.4	0.19%	26608.8	0.19%	26594.4	0.14%	26580.4	0.08%
X-n266-k58	75856	0.5%	75802.6	0.43%	75741.4	0.35%	75707.6	0.3%	75697.8	0.29%
X-n270-k35	35316.8	0.07%	35310.8	0.06%	35317.4	0.07%	35308.2	0.05%	35315.8	0.07%
X-n275-k28	21249	0.02%	21245	0%	21245	0%	21245	0%	21245	0%
X-n280-k17	33601.4	0.29%	33595.2	0.28%	33585.8	0.25%	33600.4	0.29%	33583.4	0.24%
X-n284-k15	20314	0.44%	20286.4	0.3%	20281.8	0.28%	20265.4	0.19%	20261.8	0.18%
X-n289-k60	95755.6	0.64%	95774.8	0.66%	95519.4	0.39%	95774	0.65%	95378.2	0.24%
X-n294-k50	47252.4	0.19%	47235.6	0.16%	47216	0.12%	47220.2	0.13%	47233.6	0.15%
X-n298-k31	34270.8	0.12%	34265.2	0.1%	34263	0.09%	34269.8	0.11%	34272.8	0.12%
X-n303-k21	21811.2	0.35%	21793.2	0.26%	21786.4	0.23%	21788.8	0.24%	21788.4	0.24%
X-n308-k13	25907.8	0.19%	25935.4	0.3%	25914.8	0.22%	25889.2	0.12%	25892.6	0.13%
X-n313-k71	94289.2	0.26%	94247.6	0.22%	94165.8	0.13%	94230.2	0.2%	94177.8	0.14%
X-n317-k53	78399	0.06%	78387.6	0.04%	78401.8	0.06%	78366.2	0.01%	78368.2	0.02%
X-n322-k28	29897.6	0.21%	29930.2	0.32%	29875.6	0.14%	29897.2	0.21%	29885.4	0.17%
X-n327-k20	27611	0.29%	27580.8	0.18%	27580.4	0.18%	27584	0.19%	27605.4	0.27%
X-n331-k15	31157.4	0.18%	31160.2	0.19%	31116	0.05%	31112.4	0.03%	31104.2	0.01%
X-n336-k84	140202.6	0.78%	139885	0.56%	139838	0.52%	139712.4	0.43%	139448.6	0.24%
X-n344-k43	42145.6	0.23%	42108.6	0.14%	42111.4	0.15%	42149.2	0.24%	42113.4	0.15%
X-n351-k40	25990	0.36%	25982	0.33%	25973.4	0.3%	25979.4	0.32%	25999.4	0.4%
X-n359-k29	51896.2	0.76%	52020.2	1%	51752.2	0.48%	51711.6	0.4%	51726.8	0.43%
X-n367-k17	22819.2	0.02%	22814	0%	22815.6	0.01%	22819.8	0.03%	22815.4	0.01%
X-n376-k94	147796.6	0.06%	147788.8	0.05%	147734	0.01%	147754.2	0.03%	147730.8	0.01%
X-n384-k52	66260	0.49%	66158.2	0.33%	66145.8	0.31%	66120.6	0.27%	66128.6	0.29%
X-n393-k38	38294.8	0.09%	38277.6	0.05%	38308.4	0.13%	38284.6	0.06%	38293.6	0.09%
X-n401-k29	66423.8	0.41%	66334.4	0.27%	66300.4	0.22%	66253.6	0.15%	66280.8	0.19%
X-n411-k19	19746.8	0.18%	19748.4	0.18%	19735.6	0.12%	19720	0.04%	19729.8	0.09%
X-n420-k130	107963.6	0.15%	107916.2	0.11%	107918.4	0.11%	107878.4	0.07%	107934.6	0.13%
X-n429-k61	65578.8	0.2%	65607.8	0.24%	65585.2	0.21%	65523.6	0.11%	65576.4	0.19%
X-n439-k37	36422.4	0.09%	36414.4	0.06%	36411.8	0.06%	36424.8	0.09%	36419	0.08%
X-n449-k29	55850.8	1.12%	55659	0.77%	55565	0.6%	55507	0.5%	55483.4	0.45%
X-n459-k26	24212.2	0.3%	24191.4	0.22%	24201.6	0.26%	24182.6	0.18%	24184.8	0.19%
X-n469-k138	223620	0.81%	223305.8	0.67%	223117.8	0.58%	222741.2	0.41%	222988.4	0.52%
X-n480-k70	89756.2	0.34%	89685.2	0.26%	89623.6	0.2%	89588.8	0.16%	89554.8	0.12%
X-n491-k59	66997.6	0.77%	66886.2	0.61%	66848.8	0.55%	66772.6	0.44%	66683.6	0.3%
X-n502-k39	69324	0.14%	69307.2	0.12%	69278.4	0.08%	69273.2	0.07%	69267.6	0.06%
X-n513-k21	24220.4	0.08%	24226.4	0.1%	24222	0.09%	24230.8	0.12%	24208	0.03%

⁸Naudota originali realizacija

Uždavinys	1 gija ⁹		2 gijos		4 gijos		8 gijos		16 gijos	
	Avg	Gap	Avg	Gap	Avg	Gap	Avg	Gap	Avg	Gap
X-n524-k153	154951.4	0.23%	154982.4	0.25%	154963.4	0.24%	155021.2	0.28%	154950.2	0.23%
X-n536-k96	95547.8	0.74%	95361.8	0.54%	95255.4	0.43%	95258	0.43%	95234.2	0.41%
X-n548-k50	87053	0.41%	86928	0.26%	86851.4	0.17%	86870.4	0.2%	86829.8	0.15%
X-n561-k42	42808	0.21%	42815	0.23%	42807.4	0.21%	42774.4	0.13%	42764.6	0.11%
X-n573-k30	51087.4	0.82%	51045.4	0.73%	50975	0.6%	50900.8	0.45%	50860.2	0.37%
X-n586-k159	191509.8	0.63%	191373.4	0.56%	191186.2	0.46%	190922	0.32%	190897	0.31%
X-n599-k92	109069.4	0.57%	109017.2	0.52%	109057.8	0.56%	108882.6	0.4%	108870.8	0.39%
X-n613-k62	59975	0.74%	59869.8	0.56%	59745.6	0.35%	59844.4	0.52%	59817.2	0.47%
X-n627-k43	62869.4	1.13%	62850.4	1.1%	62728.2	0.91%	62655.2	0.79%	62562.8	0.64%
X-n641-k35	64464.6	1.23%	64294	0.96%	64047.4	0.57%	64069.4	0.61%	63945.4	0.41%
X-n655-k131	106924.6	0.14%	106892.8	0.11%	106846.2	0.06%	106831	0.05%	106841.2	0.06%
X-n670-k130	147471.2	0.78%	147367.2	0.71%	147195.8	0.59%	147165.2	0.57%	147041.2	0.48%
X-n685-k75	68725	0.76%	68592.4	0.57%	68502.2	0.44%	68482.8	0.41%	68484.6	0.41%
X-n701-k44	83180.6	1.54%	83065.4	1.39%	82578.4	0.8%	82613.8	0.84%	82503	0.71%
X-n716-k35	43900.2	1.22%	43772.8	0.92%	43728.8	0.82%	43567.6	0.45%	43574	0.46%
X-n733-k159	136768	0.43%	136792	0.44%	136717.8	0.39%	136566.4	0.28%	136598.8	0.3%
X-n749-k98	78401.6	1.47%	78411.4	1.48%	78206.2	1.21%	78064.2	1.03%	77968.8	0.91%
X-n766-k71	115315.2	0.79%	115179.4	0.67%	115114.4	0.61%	114935.8	0.45%	114904.2	0.43%
X-n783-k48	73721.4	1.84%	73577	1.65%	73425.4	1.44%	73074.6	0.95%	72998.6	0.85%
X-n801-k40	74229.8	1.25%	74182.6	1.19%	73988.2	0.92%	73847	0.73%	73724.2	0.56%
X-n819-k171	159540.4	0.9%	159153.2	0.65%	158976.2	0.54%	158891.2	0.49%	158885.2	0.48%
X-n837-k142	196146	1.24%	195816.2	1.07%	195501	0.91%	195255.2	0.78%	195175.4	0.74%
X-n856-k95	89094.2	0.15%	89112.2	0.17%	89093	0.14%	89044.6	0.09%	89101	0.15%
X-n876-k59	100509.8	1.22%	100395.4	1.1%	100196.4	0.9%	100084.4	0.79%	100048.8	0.76%
X-n895-k37	54604.8	1.38%	54511.8	1.21%	54272.4	0.77%	54208	0.65%	54249.4	0.72%
X-n916-k207	332510.6	1.01%	332303.2	0.95%	332145.8	0.9%	331893.8	0.82%	331591.6	0.73%
X-n936-k151	133835	0.84%	133767	0.79%	133855.8	0.86%	133585.6	0.66%	133661.4	0.71%
X-n957-k87	86039	0.67%	85958.4	0.58%	85765.2	0.35%	85721.8	0.3%	85681.8	0.25%
X-n979-k58	120554	1.33%	120308.2	1.12%	120170.2	1%	119805.4	0.7%	119821.6	0.71%

⁹Naudota originali realizacija

Uždavinys	1 gija ¹⁰		2 gijos		4 gijos		8 gijos		16 gijos	
	Avg	Gap	Avg	Gap	Avg	Gap	Avg	Gap	Avg	Gap
X-n1001-k43	73938.4	2.19%	73748.4	1.93%	73368.4	1.4%	73202	1.17%	73232.8	1.21%

Lent. 1: Vidutinės pasiektos sprendimų kainos ir tarpas galutiniu laiko momentu

¹⁰Naudota originali realizacija

Šaltiniai

- [DR59] G. B. Dantzig ir J. H. Ramser, „The Truck Dispatching Problem“, *Management Science*, t. 6, nr. 1, p. 80–91, spal. 1959, doi: [10.1287/mnsc.6.1.80](https://doi.org/10.1287/mnsc.6.1.80).
- [Pet+23] F. Petropoulos ir kt., „Operational Research: methods and applications“, *Journal of the Operational Research Society*, t. 75, nr. 3, p. 423–617, gruodž. 2023, doi: [10.1080/01605682.2023.2253852](https://doi.org/10.1080/01605682.2023.2253852).
- [DIM22] DIMACS, „The 12th DIMACS Implementation Challenge: Vehicle Routing Problems (VRP)“, 2022 m.
- [Koo+22] W. Kool ir kt., „Hybrid genetic search for the vehicle routing problem with time windows: A high-performance implementation“, *12th dimacs implementation challenge workshop*, 2022.
- [Jia+22] S. Jiang, Z. He, W. Lin, F. Ma, ir Z. Lü, „FHCSolver: Fast hybrid CVRP solver“, *12th DIMACS implementation challenge: Vehicle routing problems*, 2022.
- [Lat25] V. Latorre, „A hybrid genetic search based approach for the generalized vehicle routing problem“, *Soft Computing*, t. 29, nr. 3, p. 1553–1566, vas. 2025a, doi: [10.1007/s00500-025-10507-0](https://doi.org/10.1007/s00500-025-10507-0).
- [Lat25] V. Latorre, „An application of a two-level genetic search for the soft-clustered vehicle routing problem“, *Evolutionary Intelligence*, t. 18, nr. 4, liep. 2025b, doi: [10.1007/s12065-025-01063-5](https://doi.org/10.1007/s12065-025-01063-5).
- [PF25] L. Perron ir V. Furnon, „OR-Tools“, [Interaktyvus]. Adresas: <https://developers.google.com/optimization/>
- [Ada+24] T. Adamo, M. Gendreau, G. Ghiani, ir E. Guerriero, „A review of recent advances in time-dependent vehicle routing“, *European Journal of Operational Research*, t. 319, nr. 1, p. 1–15, lapkr. 2024, doi: [10.1016/j.ejor.2024.06.016](https://doi.org/10.1016/j.ejor.2024.06.016).
- [Vid+12] T. Vidal, T. G. Crainic, M. Gendreau, N. Lahrichi, ir W. Rei, „A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems“, *Operations Research*, t. 60, nr. 3, p. 611–624, birž. 2012, doi: [10.1287/opre.1120.1048](https://doi.org/10.1287/opre.1120.1048).
- [Vid+14] T. Vidal, T. G. Crainic, M. Gendreau, ir C. Prins, „A unified solution framework for multi-attribute vehicle routing problems“, *European Journal of Operational Research*, t. 234, nr. 3, p. 658–673, geg. 2014, doi: [10.1016/j.ejor.2013.09.045](https://doi.org/10.1016/j.ejor.2013.09.045).
- [Vid+16] T. Vidal, N. Maculan, L. S. Ochi, ir P. H. Vaz Penna, „Large Neighborhoods with Implicit Customer Selection for Vehicle Routing Problems with Profits“, *Transportation Science*, t. 50, nr. 2, p. 720–734, geg. 2016, doi: [10.1287/trsc.2015.0584](https://doi.org/10.1287/trsc.2015.0584).
- [Vid17] T. Vidal, „Node, Edge, Arc Routing and Turn Penalties: Multiple Problems—One Neighborhood Extension“, *Operations Research*, t. 65, nr. 4, p. 992–1010, rugpj. 2017, doi: [10.1287/opre.2017.1595](https://doi.org/10.1287/opre.2017.1595).
- [Vid+21] T. Vidal, R. Martinelli, T. A. Pham, ir M. H. Hà, „Arc Routing with Time-Dependent Travel Times and Paths“, *Transportation Science*, t. 55, nr. 3, p. 706–724, geg. 2021, doi: [10.1287/trsc.2020.1035](https://doi.org/10.1287/trsc.2020.1035).

- [Vid22] T. Vidal, „Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood“, *Computers & Operations Research*, t. 140, p. 105643, bal. 2022, doi: [10.1016/j.cor.2021.105643](https://doi.org/10.1016/j.cor.2021.105643).
- [LHW25] Z. Lei, J.-K. Hao, ir Q. Wu, „Speeding up Local Optimization in Vehicle Routing with Tensor-based GPU Acceleration“. [Interaktyvus]. Adresas: <https://arxiv.org/abs/2506.17357>
- [SND23] T. Stadtler, S. Nita, ir J. Dünnweber, „A parallel hybrid genetic search for the capacitated vrp with pickup and delivery“, Ostbayerische Technische Hochschule Regensburg, 2023.
- [Mun+23] R. P. Muniasamy, S. Singh, R. Nasre, ir N. Narayanaswamy, „Effective Parallelization of the Vehicle Routing Problem“, *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '23*. ACM, liep. 2023, p. 1036–1044. doi: [10.1145/3583131.3590458](https://doi.org/10.1145/3583131.3590458).
- [YT21] P. Yelmewad ir B. Talawar, „Parallel Version of Local Search Heuristic Algorithm to Solve Capacitated Vehicle Routing Problem“, *Cluster Computing*, t. 24, nr. 4, p. 3671–3692, liep. 2021, doi: [10.1007/s10586-021-03354-9](https://doi.org/10.1007/s10586-021-03354-9).
- [Jam+25] M. Jamshidi, M. Alirezanejad, H. Motameni, ir R. Enayatifar, „A Parallel Hybrid Genetic Search for Solving the Capacitated Vehicle Routing Problem“, *International Journal of Computational Intelligence Systems*, t. 18, nr. 1, lapkr. 2025, doi: [10.1007/s44196-025-01059-0](https://doi.org/10.1007/s44196-025-01059-0).
- [Rez+24] B. Rezaei, F. Gadelha Guimaraes, R. Enayatifar, ir P. C. Haddow, „Exploring dynamic population Island genetic algorithm for solving the capacitated vehicle routing problem“, *Memetic Computing*, t. 16, nr. 2, p. 179–202, birž. 2024, doi: [10.1007/s12293-024-00412-8](https://doi.org/10.1007/s12293-024-00412-8).
- [Uch+17] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, ir A. Subramanian, „New benchmark instances for the Capacitated Vehicle Routing Problem“, *European Journal of Operational Research*, t. 257, nr. 3, p. 845–858, kovo 2017, doi: [10.1016/j.ejor.2016.08.012](https://doi.org/10.1016/j.ejor.2016.08.012).